

#####

```
##
##
##  ##  #####  #####  #  #####
##  ##  ##  ##  ##  ##  ##  ##
##  ##  ##  ##  ##  ##  ##  ##
##  ##  #####  #####  ##  ##  #####
##  ##  ##  ##  #####  ##
##  ##  ##  ##  ##  ##  ##
##  ##  ##  ##  ##  ##  ##  ##
#####  #####  #####  ##  ##  #####
```

```
#####  ##  ##  ##  #####  #####
##  ##  ##  ##  ##  ##  ##
##  ##  ##  ##  ##  ##  ##
##  ##  ##  ##  ##  ##  ##  #####
##  #####  ##  ##  ##  ##  ##
##  ##  ##  ##  ##  ##  ##  ##
##  ##  ##  ##  ##  ##  ##  ##
#####  #####  ##  #####  #####
```

#####

Interfacing routines to the Fortran incore system.

Rather than require the user access it with normal lisp functions and run the risk of the implementation of the virtual machine changing, a set of data accessing primitives are provided. This guide describes the specifications for the functions that are provided with the translator.

(c) 1979 Massachusetts Institute of Technology

The Fortran Virtual Machine

A number of conventions are adhered to in all translations of Fortran programs in order to assure that the proper effect of these routines is carried through during the translation.

Data Storage

In the runtime environment of a translated Fortran routine, there will exist a set of library routines for managing the data areas used by the translation. Essentially, all memory references in the source Fortran will translate into references to locations in a Fortran virtual machine via the functions FGET and FPUT. The calling conventions for these functions will be described later. Essentially, however, these operations operate on un-typed data stored in a MacLISP FIXNUM array. Type information is propagated explicitly in the translation. Type coercions are also made explicit.

Calling Conventions

In all but one case, all formal parameters of a function may be expected to be bound not to a value, but to a FIXNUM which is the offset in the Fortran virtual machine of the value being passed. These FIXNUM's may be thought of as addresses and the addresses serve to give calls to these routines a Call-By-Reference flavor as called for by Fortran. Thus if X is a formal parameter of type REAL, then (FGET FLONUM X) will retrieve its value and (FPUT FLONUM X <flonum-expression>) will store a value into X.

The exception to the rule about formal parameters being FIXNUM's is the case of a formal parameter being used in a functional position (and not having been dimensioned) in the source Fortran. This will expand in the compiler into (FUNCALL X ...) and will expect that X's value is a lisp symbol which is Fortran-Callable (ie, expects arguments which are indexes into the Fortran array rather than actual values).

The caller of a routine which expects to FUNCALL one of its args must have declared the name of the routine which it will pass as EXTERNAL and if it has done so, the arg will be passed to the routine as a quoted symbol. (See figure 1.)

Figure 1. Crosscut of a Translation

```
PROGRAM MAIN
EXTERNAL F
CALL FOO (F,I)
...
END
SUBROUTINE FOO (FUN,I)
I=FUN(I)
RETURN
END
```

Might translate to something like

```
(DEFUN MAIN () (PROG () (FOO 'F <location-of-I>) ...))
(DEFUN FOO (FUN I) (FPUT FIXNUM (FUNCALL FUN I)))
```

Runtime Support

This section is devoted to information about functions that are available to user interfaces at runtime. The source for this support lives in the file MC:TNP;INCORE > and the compiled form is in MC:TNP;INCORE FASL. These will be loaded automatically by code output from the translator.

Hand coded routines should include the information contained in the file MC:TNP;INSERT RUNTIM at the head of any file. The information in that file will be kept up to date with necessary procedures for properly initializing the Fortran runtime environment.

(FORTRAN\$INITIALIZE <flag>)

This is a SUBR which initializes the Fortran virtual machine, creating all the necessary.

<flag> ::= a boolean value (T or NIL)

If the Fortran environment has not been initialized,
then it will be initialized regardless of the value of <flag>
else if <flag> = NIL
then the Fortran environment will not be re-initialized
else the Fortran environment will be re-initialized

The function FORTRAN\$INITIALIZE returns:

INITIALIZED ; if the database is initialized for the first time
RE-INITIALIZED ; if re-initialization of the database has been done
or NIL ; if initialization had already been done and
; re-initialization was not requested.

(FORTRAN\$ALLOCATE <name> <size> <common?>)

This is a SUBR that will allocate space in the Fortran array that is reserved for use solely by those persons with pointers to <name>. Gensym's are advised as <name>'s for user-packages wishing to tap into this facility.

<name> ::= a lisp symbol.
<size> ::= a fixnum indicating the number of storage slots desired in the virtual machine.
<common?> ::= a flag that is non-NIL if the object is common.
If flag = NIL
then if a storage area by that name already exists
then an error will be generated
else a new area will be allocated with given name
else if a storage area by that name already exists
then if the new area is not larger than the old one
then the two areas will share with each other
else an error will be generated
else a new area will be allocated with given name

The FORTRAN\$ALLOCATE command returns:

(<initial-location> . <terminal-location>)

which is a CONS of the two virtual machine addresses which bound the space requested, inclusive. (This information need not be held onto if there is not immediate use for it -- it is stored in a location globally accessible in the Lisp.

(FORTRAN\$GET-BLOCK-INFO <name>)

This is a SUBR which returns information about a <name> that space has been allocated for.

<name> ::= any lisp symbol

The FORTRAN\$GET-BLOCK-INFO command returns:

NIL ; if no space is on
; record under the given name
or
(<initial-location> . <size>) ; if space has been allocated for it.

(FORTRAN\$GC <name>)

This is a SUBR which removes information about a given <name>'d space from the Fortran virtual machine, returning it to the free list.

<name> ::= any lisp symbol.

The function FORTRAN\$GC returns T if successful, NIL if there was no info on <name> to remove.

(FORTRAN\$OFFSET <name>)

This is a SUBR which returns information the initial location of space for <name> in the virtual machine.

<name> ::= any lisp symbol

The FORTRAN\$OFFSET command returns:

NIL ; if no space is on record under the given name
or
<initial-location> ; if space has been allocated for it.

Useful Free Variables

\$INITIALFORTRANWORKAREA

Contains the default size of the initial Fortran memory. If this variable is not set, a value of 1000. will be used instead. This allows the user to reserve ahead of time a large (or small) area of space if he has some advance knowledge about the amount of space that will be required by routines to be loaded later.

\$FORTRANDYNAMALLOC

Says whether to be chincy about space. If set to T tells lisp not to rearray the fortran array to a larger size unless it absolutely has to and then to do it in increments only of what's needed. The default is for the array to be increased in size in units of 1000 words any time there is an array overflow to avoid having to copy the array often due to small requests.

\$FORTRANKEEPSYMTAB

If non-NIL (default) will cause Fortran symbol tables to be retained in the runtime environment.

Macro Support

This section is devoted to information about functions that are available to user interfaces at runtime. The source for this support lives in the file MC:TNP;FMACRO > and the compiled form is in MC:TNP;FMACRO FASL. These will be loaded automatically by code output from the translator.

Hand coded routines should include the information contained in the file MC:TNP;INSERT COMPLR at the head of any file. The information in that file will be kept up to date with necessary procedures for properly initializing the Fortran runtime environment.

Subroutine Calling Macros

(FCALL <function-name> <object1> <object2> ... <objectN>)

<function-name> ::= an unquoted function-name to be called.

<object> ::= an object to be changed to Fortran representation
and passed a la Call-by-Value/Return to <function-name>

If <object> is a SYMBOL, then its value will be passed to the called routine
and the symbol's value will be SETQ'd to the new value upon return.

If <object> is a one-length list, then its CAR is the name of an array
to be used. Its array property will be looked up and that will be
copied into the Fortran array before the call and the Fortran virtual
machine copied back into the Lisp array after the call.

If <object> is a list of numbers or any nesting of lists of numbers in a
matrix-like organization of arbitrary dimensions, then the numbers will
be linearized in the manner defined by Fortran for conversion from
multi-dimensional form to one-dimensional form, and then copied into the
Fortran array. Upon return from the routine, the list will be
reconstructed from the Fortran machine's memory and displaced back into
the old list.

If <object> is a two-length list beginning with the symbol QUOTE, then
the second object must be a function name. This will be passed as
a quoted object to the Fortran function and is the correct syntax
to use for passing an external function in. (Note that unless you have
removed the FORTRAN-CALL declaration for that formal parameter
from the receiving function, this function must be Fortran-callable.)
Sample usage: (FCALL MYFUN 'EXTFN)

FCALL expands into a call to *FCALL which is the LSUBR version of
this operator.

Runtime support for FCALL

(*FCALL <function-name> <object1> <object2> ...<objectN>)

The *FCALL subr will autoloading in any environment that contains a naturally translated Fortran routine or can be manually loaded from the file MC:TNP;FCALL FASL.

<function-name> ::= must evaluate to the name of a function-name to be APPLY'd to the Fortran virtual machine addresses of a set of data.

<object> ::= must evaluate to an object to be changed to Fortran representation (the pointer to which will be passed to <function-name> in a Call-by-Value/Return fashion.

If <object> is a lisp number, it will be copied into the Fortran array. It will not be (cannot be due to the lisp data formalism) altered in value upon return.

If <object> is a list of numbers or any nesting of lists of numbers in a matrix-like organization of arbitrary dimensions (in row major order), then the numbers will be linearized (into Fortran column major order format) Upon return from the routine, the list will be reconstructed in row major order producing necessary side-effects on the inputs.

If <object> is a lisp array (stored in row major order), it will be copied appropriately (in column major order) into the Fortran virtual memory area. Upon return from the routine, the values will be copied back to their respective locations in the input array from the virtual memory.

The *FCALL subr will autoloading in any environment that contains a naturally translated Fortran routine or can be manually loaded from the file MC:TNP;FCALL FASL.

MACSYMA Compatibility Macro

(MFCALL <function-name> <object1> <object2> ... <objectN>)

<function-name> ::= an unquoted function-name to be called.

<object> ::= an object to be changed to Fortran representation
and passed a la Call-by-Value/Return to <function-name>

All <object>'s will be Macsyma <eval>'d, converted to standard lisp format (see below) and passed to *FCALL. Although the objects themselves may be Macsyma structures, the elements must be lisp FIXNUM's or FLONUM's. BIGNUM's and BFLOAT's are not valid input objects.

If <object> evaluates to a Macsyma matrix, it will be converted to a normal lisp list containing a set of lists representing each of the rows (standard input format for *FCALL). Side-effects due to Fortran program execution will affect the Macsyma input matrix.

If <object> evaluates to a Macsyma list, it will be converted to a lisp list. Side-effects due to Fortran program execution will affect the Macsyma input list.

If <object> evaluates to a number, it will be used as is. Side-effects due to Fortran program execution will NOT affect the value of the input constant (or the variable which contained it).

NOTE: This latter case does NOT produce the same effect as the FCALL macro for lisp code which CAN affect the value of an input symbol which contains a constant number.

Storage Accessing Macros

Macros are provided for storage and retrieval in the Fortran virtual machine space...

(FPUT <type> <location> <value>) ; Stores a value in the machine

(FGET <type> <location>) ; Gets a value from the machine

<type> ::= an un-evaluated form which is an acceptable first arg to
to ARRAYCALL for storing into a number array (ie, FIXNUM
or FLONUM)

<location> ::= this must evaluate to a FIXNUM which is the virtual
machine address which is to be accessed.

<value> ::= this must evaluate to a FIXNUM or a FLONUM. (Note that
Holleriths and Logical variables are treated specially
in the translation phase and will map into FIXNUM's.

Other useful macros...

(IF <condition> <action>)

This macro expects <condition> to return a fixnum. If the fixnum is
non-zero, then <action> will be executed. (See documentation below
on functions that return Fortran truth values.)

Predicates and Truth Values

Fortran predicates return 777777,,777777 (a word of all ones) for True and 000000,,000000 (a word of all zeros) for False. For compatibility with the lisp world, a set of macros is provided that helps jump back and forth between Lisp truth values and Fortran truth values.

Convention:

Macros with a leading "*" in the name expect Fortran inputs.
Macros with a trailing "*" give Fortran outputs.

Note that since arithmetic operators (EQUAL, NE, LE, ...) expect numbers as inputs and numbers are the same in both Fortran and Lisp representation, it makes no difference whether the operator is chosen with a leading "*" or not. Pairs of operators such as: [*GT,GT] or [GT*,*GT*] are fully interchangeable since they do not differ in output type. Both operators are provided to add generality to the naming convention.

Boolean Operators

Function Name	Operation	Input Type	Output Type

AND	AND	Fortran	Fortran
AND*	AND	Lisp	Fortran
AND	AND	Lisp	Lisp
*AND	AND	Fortran	Lisp
OR	OR	Fortran	Fortran
OR*	OR	Lisp	Fortran
OR	OR	Lisp	Lisp
*OR	OR	Fortran	Lisp
NOT	NOT	Fortran	Fortran
NOT*	NOT	Lisp	Fortran
NOT	NOT	Lisp	Lisp
*NOT	NOT	Fortran	Lisp

Arithmetic Comparison

Function Name	Operation	Input Type	Output Type
EQUAL	EQUAL	(Fortran)	Fortran
EQUAL*	EQUAL	(Lisp)	Fortran
EQUAL	EQUAL	(Lisp)	Lisp
*EQUAL	EQUAL	(Fortran)	Lisp
NE	NE	(Fortran)	Fortran
NE*	NE	(Lisp)	Fortran
NE	NE	(Lisp)	Lisp
*NE	NE	(Fortran)	Lisp
LT	LESSP	(Fortran)	Fortran
LT*	LESSP	(Lisp)	Fortran
LT	LESSP	(Lisp)	Lisp
*LT	LESSP	(Fortran)	Lisp
LE	LE	(Fortran)	Fortran
LE*	LE	(Lisp)	Fortran
LE	LE	(Lisp)	Lisp
*LE	LE	(Fortran)	Lisp
GE	GE	(Fortran)	Fortran
GE*	GE	(Lisp)	Fortran
GE	GE	(Lisp)	Lisp
*GE	GE	(Fortran)	Lisp
GT	GREATERP	(Fortran)	Fortran
GT*	GREATERP	(Lisp)	Fortran
GT	GREATERP	(Lisp)	Lisp
*GT	GREATERP	(Fortran)	Lisp